



MASTER INTELLIGENCE ARTIFICIELLE
TRAVAUX DIRIGÉS — MACHINE LEARNING

TD N° 04

Régression Linéaire & Régression Logistique

Durée : 2 heures 30 minutes	Niveau : Débutant – Intermédiaire
Datasets : California Housing + BankChurners	Outils : sklearn, numpy, matplotlib, seaborn

Objectifs du TD

- Implémenter le **Gradient Descent** from scratch pour comprendre l'optimisation
- Maîtriser la **Régression Linéaire** simple et multiple avec sklearn
- Comprendre la **Régression Logistique** : fonction sigmoïde, log-vraisemblance, frontière de décision
- **Interpréter les coefficients** : sens, magnitude, odds ratio
- Visualiser et analyser les **frontières de décision** dans l'espace des features
- Comparer l'impact de la **régularisation L1/L2** sur les coefficients
- Évaluer les modèles avec des métriques adaptées : RMSE (régression), F1/AUC (classification)

Pr. BEN LAHMAR EL Habib · Semestre 2024–2025



Contexte et Rappels Théoriques

Ce TD couvre deux algorithmes fondamentaux du ML supervisé : la **Régression Linéaire** (prédire une valeur continue) et la **Régression Logistique** (classer en deux catégories ou plus). Les deux partagent le même mécanisme d'optimisation : la descente de gradient.

📖 Rappel de cours — Régression Linéaire — Principe

Modèle : $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n = \mathbf{X}\theta$

- **Objectif** : minimiser l'erreur quadratique moyenne — $MSE(\theta) = (1/m) \sum (\hat{y}_i - y_i)^2$
- **Solution analytique** (équation normale) : $\theta = (X^T X)^{-1} X^T y$ — coûteuse si n_{features} grand
- **Solution itérative** : Gradient Descent — $\theta := \theta - \alpha \nabla MSE(\theta)$
- **Hypothèses** : linéarité, homoscedasticité, indépendance des résidus, normalité des résidus
- **Métriques** : MAE, RMSE (même unité que y), R^2 (proportion de variance expliquée)

📖 Rappel de cours — Régression Logistique — Principe

Modèle : $P(y=1 | \mathbf{x}) = \sigma(\mathbf{X}\theta) = 1 / (1 + e^{-(\mathbf{X}\theta)})$

- **Pas une régression** au sens classique : c'est un **classifieur** qui estime une probabilité
- **Fonction sigmoïde** $\sigma(z) \in]0,1[$: transforme un score linéaire en probabilité
- **Objectif** : maximiser la log-vraisemblance (= minimiser la log-loss)
- **Décision** : $P(y=1 | \mathbf{x}) > \text{seuil}$ (défaut 0.5) $\rightarrow$ classe 1, sinon $\rightarrow$ classe 0
- **Frontière de décision** : $\mathbf{X}\theta = 0 \rightarrow$ hyperplan linéaire dans l'espace des features
- **Interprétation des coefs** : $\exp(\theta_j)$ = odds ratio — multiplication des cotes pour +1 unité de x_j

📖 Rappel de cours — Gradient Descent — Les 3 variantes

Variante	Données utilisées	Vitesse	Avantage
Batch GD	Tout le dataset	Lent	Convergence stable
Stochastic GD (SGD)	1 exemple à la fois	Très rapide	Fonctionne en ligne
Mini-batch GD	32–256 exemples	Équilibré ✓	Compromis optimal

- **Learning rate α** trop grand $\rightarrow$ divergence / trop petit $\rightarrow$ convergence très lente
- **Règle empirique** : tester $\alpha \in \{0.001, 0.01, 0.1\}$ et tracer la courbe de loss

Chargez les bibliothèques et préparez les données avant de commencer :



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import (LinearRegression, LogisticRegression,
                                   Ridge, Lasso)
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (mean_squared_error, r2_score, mean_absolute_error,
                              classification_report, confusion_matrix,
                              roc_auc_score, roc_curve, ConfusionMatrixDisplay)
from sklearn.pipeline import Pipeline
import warnings ; warnings.filterwarnings('ignore')

# Dataset 1 – Régression (California Housing)
housing = fetch_california_housing(as_frame=True)
df_h = housing.frame
print(f'California Housing : {df_h.shape}')

# Dataset 2 – Classification (BankChurners, depuis TD 2)
raw = pd.read_csv('BankChurners.csv')
df_b = raw[raw.columns[:-2]].copy()
df_b['churn'] = (df_b['Attrition_Flag']=='Attrited Customer').astype(int)
print(f'BankChurners : {df_b.shape}')
```



Partie 1 — Gradient Descent Implémenté from Scratch

Durée estimée : 30 minutes | Implémenter l'algorithme core avant de l'utiliser via sklearn — la vraie compréhension.

Q 1.1

Gradient Descent Batch — implémentation NumPy

Consigne

Implémentez le Gradient Descent Batch pour la régression linéaire en NumPy. Tracez la courbe de convergence de la MSE et comparez avec la solution analytique.

```
# Génération d'un dataset synthétique simple pour la visualisation
np.random.seed(42)
X_syn = 2 * np.random.rand(200, 1)
y_syn = 4 + 3 * X_syn.ravel() + np.random.randn(200) # y = 4 + 3x + bruit

# — IMPLÉMENTATION GRADIENT DESCENT BATCH —
def gradient_descent(X, y, alpha=0.01, n_iter=1000):
    """Régression linéaire par descente de gradient batch."""
    m = len(y)
    X_b = np.c_[np.ones((m, 1)), X] # Ajouter biais x0=1
    theta = np.zeros(X_b.shape[1]) # Initialisation à 0
    mse_history = []

    for iteration in range(n_iter):
        # Prédiction
        y_hat = X_b @ theta
        # Erreurs
        errors = y_hat - y
        # Gradient :  $\partial \text{MSE} / \partial \theta = (2/m) X^T (X\theta - y)$ 
        gradient = (2/m) * X_b.T @ errors
        # Mise à jour des poids
        theta = theta - alpha * gradient
        # Enregistrer la MSE
        mse = np.mean(errors**2)
        mse_history.append(mse)

    return theta, mse_history

# Entraînement avec différents learning rates
alphas = [0.001, 0.01, 0.1]
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for alpha in alphas:
    theta, history = gradient_descent(X_syn, y_syn, alpha=alpha, n_iter=500)
```



```
axes[0].plot(history, label=f'α={alpha}')

axes[0].set_xlabel('Itération') ; axes[0].set_ylabel('MSE')
axes[0].set_title('Convergence du Gradient Descent', fontweight='bold')
axes[0].legend() ; axes[0].set_yscale('log')

# Meilleur alpha - visualisation de la droite de régression
best_theta, _ = gradient_descent(X_syn, y_syn, alpha=0.1, n_iter=500)
x_plot = np.linspace(0, 2, 100)
y_gd = best_theta[0] + best_theta[1] * x_plot

# Solution analytique :  $\theta = (X^T X)^{-1} X^T y$ 
X_b = np.c_[np.ones((200,1)), X_syn]
theta_analytic = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y_syn
y_analytic = theta_analytic[0] + theta_analytic[1] * x_plot

axes[1].scatter(X_syn, y_syn, alpha=.5, s=20, label='Données')
axes[1].plot(x_plot, y_gd, color='#1A7A6E', lw=2.5, label=f'GD : y={best_theta[0]:.2f}+{best_theta[1]:.2f}x')
axes[1].plot(x_plot, y_analytic, color='#9C3D38', lw=2, ls='--', label=f'Analytique : y={theta_analytic[0]:.2f}+{theta_analytic[1]:.2f}x')
axes[1].set_title('GD vs Solution Analytique', fontweight='bold')
axes[1].legend()
plt.tight_layout() ; plt.show()

print(f'Vrais paramètres :  $\theta_0=4.00, \theta_1=3.00$ ')
print(f'Gradient Descent :  $\theta_0={best\_theta[0]:.3f}, \theta_1={best\_theta[1]:.3f}$ ')
print(f'Solution analytique :  $\theta_0={theta\_analytic[0]:.3f}, \theta_1={theta\_analytic[1]:.3f}$ ')
```

Questions d'analyse

1. Avec $\alpha=0.1$ et 500 itérations, les paramètres convergent-ils vers les vraies valeurs ($\theta_0=4, \theta_1=3$) ? Quelle est l'erreur résiduelle ?
2. Que se passe-t-il avec $\alpha=0.5$ ou $\alpha=1.0$? Testez et expliquez visuellement le phénomène.
3. La solution analytique est-elle exacte ? Pourquoi la solution numérique (GD) y converge mais ne l'atteint pas exactement ?
4. **Complexité calculatoire** : la solution analytique est en $O(n^3)$ (inversion de matrice). Pour un dataset avec 1 million d'exemples et 100 features, laquelle des deux méthodes choisiriez-vous ?

Q 1.2

Influence du learning rate — analyse expérimentale

```
# Test systématique de learning rates
test_alphas = [0.0001, 0.001, 0.01, 0.05, 0.1, 0.5]
```



```
results = []

for alpha in test_alphas:
    theta, history = gradient_descent(X_syn, y_syn,
                                      alpha=alpha, n_iter=300)

    final_mse = history[-1]
    converged = final_mse < 1.5 # Seuil de convergence
    results.append({
        'alpha': alpha,
        'final_mse': round(final_mse, 4),
        'convergé': '✓' if converged else 'X diverge',
        'theta_0': round(theta[0], 3),
        'theta_1': round(theta[1], 3),
    })

print('=== IMPACT DU LEARNING RATE ===')
print(pd.DataFrame(results).to_string(index=False))
```

Questions d'analyse

5. À partir de quel learning rate observez-vous une divergence ? Comment se manifeste-t-elle sur la courbe de MSE ?
6. Proposez une méthode automatique pour sélectionner un bon learning rate (hint : regardez la courbe de loss sur les 10 premières itérations avec des alphas en progression géométrique).



Partie 2 — Régression Linéaire avec Sklearn

Durée estimée : 30 minutes | Dataset California Housing — prédire le prix médian des maisons (MedHouseVal).

📖 Présentation du Dataset California Housing

- **20 640 blocs de recensement** californiens — features agrégées au niveau quartier
- **Features :** MedInc (revenu médian), HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude
- **Cible :** MedHouseVal — prix médian des maisons en centaines de milliers de dollars
- **Particularité :** MedHouseVal est plafonnée à 5.0 (\$500 000) — crée un artefact visible sur les histogrammes

Q 2.1

Régression Linéaire Simple — MedInc → MedHouseVal

Consigne

Entraînez une régression linéaire simple sur la feature la plus corrélée avec la cible. Visualisez la droite de régression et analysez les résidus.

```
# Exploration initiale
print('Corrélation avec MedHouseVal :')
corr = df_h.corr()['MedHouseVal'].drop('MedHouseVal').sort_values(ascending=False)
print(corr.round(3))

# Régression simple : MedInc → MedHouseVal
X_simple = df_h[['MedInc']].values
y_house = df_h['MedHouseVal'].values

X_tr, X_te, y_tr, y_te = train_test_split(
    X_simple, y_house, test_size=0.2, random_state=42)

lr_simple = LinearRegression()
lr_simple.fit(X_tr, y_tr)
y_pred_simple = lr_simple.predict(X_te)

print(f'Coefficient ( $\theta_1$ ) : {lr_simple.coef_[0]:.4f}')
print(f'Intercept ( $\theta_0$ ) : {lr_simple.intercept_:.4f}')
print(f' $R^2$  : {r2_score(y_te, y_pred_simple):.4f}')
print(f'RMSE : {np.sqrt(mean_squared_error(y_te, y_pred_simple)):.4f}')
print(f'MAE : {mean_absolute_error(y_te, y_pred_simple):.4f}')
```



```
# Visualisation
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1. Droite de régression
x_line = np.linspace(X_simple.min(), X_simple.max(), 100).reshape(-1,1)
axes[0].scatter(X_te, y_te, alpha=.2, s=10, color='#1A7A6E')
axes[0].plot(x_line, lr_simple.predict(x_line), color='#9C3D38', lw=2.5)
axes[0].set_xlabel('MedInc (revenu médian)') ; axes[0].set_ylabel('MedHouseVal')
axes[0].set_title('Régression Simple', fontweight='bold')

# 2. Résidus vs prédictions
residuals = y_te - y_pred_simple
axes[1].scatter(y_pred_simple, residuals, alpha=.2, s=10, color='#5B3D8F')
axes[1].axhline(0, color='red', ls='--', lw=1.5)
axes[1].set_xlabel('Valeurs prédites') ; axes[1].set_ylabel('Résidus')
axes[1].set_title('Analyse des Résidus', fontweight='bold')

# 3. Distribution des résidus
sns.histplot(residuals, kde=True, ax=axes[2], color='#B07D3A')
axes[2].set_xlabel('Résidus') ; axes[2].set_title('Distribution des Résidus',
fontweight='bold')
plt.tight_layout() ; plt.show()
```

📌 Rappel de cours — Lecture des métriques de régression

- **$R^2 \in [0,1]$** : proportion de variance expliquée. $R^2=0.6 \rightarrow$ le modèle explique 60% de la variabilité de y
- **RMSE** : erreur en même unité que y . Sensible aux grandes erreurs (pénalise le carré)
- **MAE** : erreur absolue moyenne. Plus robuste aux outliers que RMSE
- **Analyse des résidus** : doivent être aléatoires (pas de pattern) et normalement distribués
- **Pattern en résidus** = modèle trop simple, relation non-linéaire non capturée

Questions d'analyse

7. Interprétez le coefficient θ_1 dans le contexte métier. Une augmentation de 1 unité de MedInc correspond à quelle variation de MedHouseVal ?
8. Le graphique des résidus vs prédictions montre-t-il un pattern (ex : forme en entonnoir) ? Qu'est-ce que cela indique sur les hypothèses de la régression linéaire ?
9. Observez-vous un plafond dans les données à MedHouseVal = 5.0 ? Comment cet artefact affecte-t-il votre modèle et vos métriques ?



Q 2.2

Régression Linéaire Multiple — toutes les features

Consigne

Entraînez une régression multiple avec toutes les features numériques. Comparez les métriques avec la régression simple et interprétez les coefficients.

```
# Features numériques du dataset California Housing
feature_cols = [c for c in df_h.columns if c != 'MedHouseVal']
X_multi = df_h[feature_cols].values
y_house = df_h['MedHouseVal'].values

X_tr, X_te, y_tr, y_te = train_test_split(
    X_multi, y_house, test_size=0.2, random_state=42)

# Pipeline : Scaling + LinearRegression
pipe_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])
pipe_lr.fit(X_tr, y_tr)
y_pred_multi = pipe_lr.predict(X_te)

print('=== RÉGRESSION MULTIPLE ===')
print(f'R² : {r2_score(y_te, y_pred_multi):.4f}')
print(f'RMSE : {np.sqrt(mean_squared_error(y_te, y_pred_multi)):.4f}')

# Afficher les coefficients standardisés
coefs = pd.Series(
    pipe_lr.named_steps['model'].coef_,
    index=feature_cols
).sort_values(key=abs, ascending=False)

print('\n=== COEFFICIENTS STANDARDISÉS (impact par  $\sigma$ ) ===')
for feat, coef in coefs.items():
    direction = '↑ prix' if coef > 0 else '↓ prix'
    print(f' {feat:20s} : {coef:+.4f} ({direction})')

# Barplot des coefficients
fig, ax = plt.subplots(figsize=(10, 5))
colors = ['#1A7A6E' if v > 0 else '#9C3D38' for v in coefs.values]
ax.barh(coefs.index[::-1], coefs.values[::-1], color=colors[::-1], edgecolor='white')
ax.axvline(0, color='black', lw=1)
ax.set_xlabel('Coefficient standardisé') ; ax.set_title('Impact des Features sur MedHouseVal', fontweight='bold')
plt.tight_layout() ; plt.show()
```



Questions d'analyse

10. Le R^2 a-t-il augmenté par rapport à la régression simple ? De combien ? Cette amélioration justifie-t-elle la complexité supplémentaire ?
11. Quelle feature a le **plus grand coefficient positif** (= plus grande influence sur le prix) ? Quel est son sens métier ?
12. La feature **AveBedrms** (nombre moyen de chambres) a-t-elle un coefficient positif ou négatif ? Ce résultat vous surprend-il ? Comment l'expliquer ?
13. Pourquoi est-il important d'utiliser des **coefficients standardisés** pour comparer l'importance des features (et non les coefficients bruts) ?



Partie 3 — Régression Logistique pour le Churn

Durée estimée : 40 minutes | Retour sur BankChurners — prédire le churn avec la Régression Logistique.

Fonction Sigmoidale : transforme un score linéaire en probabilité

$$\sigma(z) = 1 / (1 + e^{(-z)}) \quad \text{avec } z = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

Q 3.1

Préparation des données et premier modèle logistique

Consigne

Préparez le dataset BankChurners (encodage + normalisation) et entraînez un premier modèle de Régression Logistique. Évaluez-le avec des métriques adaptées au déséquilibre de classes.

```
# — Préparation BankChurners (résumé TD 2) —————
# Encodage des variables catégorielles
df_enc = df_b.copy()
df_enc['Gender'] = df_enc['Gender'].map({'M':1,'F':0})
df_enc['Marital_Status'] = df_enc['Marital_Status'].map(
    {'Single':0,'Married':1,'Divorced':2,'Unknown':-1})

edu_order = ['Unknown','Uneducated','High School',
             'College','Graduate','Post-Graduate','Doctorate']
inc_order = ['Unknown','Less than $40K','$40K - $60K',
             '$60K - $80K','$80K - $120K','$120K +']
df_enc['Education_Level'] = df_enc['Education_Level'].map(
    {v:i for i,v in enumerate(edu_order)})
df_enc['Income_Category'] = df_enc['Income_Category'].map(
    {v:i for i,v in enumerate(inc_order)})

# OHE pour Card_Category
df_enc = pd.get_dummies(df_enc, columns=['Card_Category'],
                       prefix='card', drop_first=True, dtype=int)

num_cols = ['Customer_Age','Credit_Limit','Months_on_book',
            'Total_Relationship_Count','Months_Inactive_12_mon',
            'Contacts_Count_12_mon','Total_Revolving_Bal',
            'Avg_Open_To_Buy','Total_Trans_Amt','Total_Trans_Ct',
            'Avg_Utilization_Ratio','Total_Amt_Chng_Q4_Q1',
            'Total_Ct_Chng_Q4_Q1']
cat_cols = ['Gender','Marital_Status','Education_Level',
            'Income_Category'] + [c for c in df_enc.columns if c.startswith('card_')]
```



```
X = df_enc[num_cols + cat_cols]
y = df_enc['churn']

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Pipeline : Scaling + LogisticRegression
pipe_log = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000, random_state=42))
])
pipe_log.fit(X_tr, y_tr)
y_pred_log = pipe_log.predict(X_te)
y_prob_log = pipe_log.predict_proba(X_te)[:, 1]

print('=== ÉVALUATION - RÉGRESSION LOGISTIQUE ===')
print(classification_report(y_te, y_pred_log,
    target_names=['Existing (0)', 'Attrited (1)']))
print(f'AUC-ROC : {roc_auc_score(y_te, y_prob_log):.4f}')
```

Questions d'analyse

14. Le modèle prédit-il bien les deux classes ? Comparez le **recall** de la classe 1 (Attrited) avec celui de la classe 0. Quel est le problème ?
15. **Accuracy vs F1** : calculez l'accuracy en supposant que le modèle prédit toujours 'Existing Customer'. Obtenez-vous ~83,9% ? Pourquoi cette accuracy est-elle trompeuse ?
16. L'AUC-ROC obtenu est-il satisfaisant ? Selon les seuils vus en cours, comment qualifieriez-vous ce modèle ?

Q 3.2

Courbe ROC et optimisation du seuil de décision

Consigne

Tracez la courbe ROC et analysez l'impact du seuil de décision sur le compromis précision/rappel. Trouvez le seuil optimal selon le contexte métier.

```
# Courbe ROC
fpr, tpr, thresholds = roc_curve(y_te, y_prob_log)
auc = roc_auc_score(y_te, y_prob_log)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# ROC Curve
```



```
axes[0].plot(fpr, tpr, color='#1A7A6E', lw=2.5,
             label=f'ROC Curve (AUC = {auc:.3f})')
axes[0].plot([0,1],[0,1], 'k--', lw=1, label='Classifieur aléatoire')
axes[0].fill_between(fpr, tpr, alpha=.1, color='#1A7A6E')
# Point optimal (Youden index : max TPR - FPR)
youden_idx = np.argmax(tpr - fpr)
axes[0].scatter(fpr[youden_idx], tpr[youden_idx], color='#9C3D38',
               s=120, zorder=5, label=f'Seuil opt. = {thresholds[youden_idx]:.2f}')
axes[0].set_xlabel('Taux de Faux Positifs (FPR)')
axes[0].set_ylabel('Taux de Vrais Positifs (TPR)')
axes[0].set_title('Courbe ROC', fontweight='bold')
axes[0].legend()

# Impact du seuil sur Precision et Recall
from sklearn.metrics import precision_recall_curve
precision, recall, pr_thresh = precision_recall_curve(y_te, y_prob_log)
axes[1].plot(pr_thresh, precision[:-1], label='Précision', color='#5B3D8F', lw=2)
axes[1].plot(pr_thresh, recall[:-1], label='Rappel', color='#B07D3A', lw=2)
axes[1].axvline(0.5, color='gray', ls='--', lw=1, label='Seuil par défaut (0.5)')
axes[1].axvline(thresholds[youden_idx], color='#9C3D38', ls='--', lw=1.5,
               label=f'Seuil optimal ({thresholds[youden_idx]:.2f})')
axes[1].set_xlabel('Seuil de décision')
axes[1].set_ylabel('Score') ; axes[1].legend()
axes[1].set_title('Précision et Rappel vs Seuil', fontweight='bold')
plt.tight_layout() ; plt.show()

# Prédiction avec seuil optimal
seuil_opt = thresholds[youden_idx]
y_pred_opt = (y_prob_log >= seuil_opt).astype(int)
print(f'\n=== SEUIL OPTIMAL = {seuil_opt:.3f} ===')
print(classification_report(y_te, y_pred_opt,
                           target_names=['Existing (0)', 'Attrited (1)']))
```

📌 Rappel de cours — Choix du seuil de décision selon le contexte métier

- **Seuil 0.5 (défaut)** : équilibre précision/rappel — rarement optimal sur données déséquilibrées
- **Seuil bas (ex : 0.3)** : augmente le Rappel → détecte plus de churns, mais plus de fausses alarmes
- **Seuil haut (ex : 0.7)** : augmente la Précision → moins de fausses alarmes, mais manque des churns
- **Contexte bancaire** : Manquer un churn (FN) est généralement plus coûteux qu'une fausse alarme (FP) → baisser le seuil pour maximiser le Rappel
- **Indice de Youden** = $\max(\text{TPR} - \text{FPR})$: seuil qui maximise la séparation ROC

Questions d'analyse

17. Avec le seuil par défaut (0.5), quel est le **Rappel de la classe Attrited** ? Et avec le seuil optimal ?



18. En contexte bancaire, le Directeur Commercial vous demande de « ne pas manquer plus de 1 churning sur 5 » — soit un Rappel minimum de 80%. Quel seuil devez-vous utiliser ? Utilisez la courbe PR pour le déterminer.
19. Quel est le compromis (trade-off) si vous abaissez le seuil de 0.5 à 0.3 : comment évoluent la Précision et le Rappel ?

Q 3.3

Interprétation des coefficients et Odds Ratios

Consigne

Extrayez et interprétez les coefficients de la Régression Logistique. Calculez les Odds Ratios pour donner une interprétation métier concrète.

```
# Extraction des coefficients
coefs_log = pd.Series(
    pipe_log.named_steps['model'].coef_[0],
    index=X_tr.columns
).sort_values(key=abs, ascending=False)

# Odds Ratios = exp(coefficient)
odds_ratios = np.exp(coefs_log)

# Tableau récapitulatif
coef_df = pd.DataFrame({
    'Coefficient': coefs_log.round(4),
    'Odds Ratio': odds_ratios.round(4),
    'Interprétation': odds_ratios.apply(
        lambda or_: f'×{or_:.2f} risque' if or_>1 else f'÷{1/or_:.2f} risque'
    )
})
print(coef_df.head(15).to_string())

# Visualisation des top features
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

top_n = 12
top_coefs = coefs_log.head(top_n)
colors_c = ['#9C3D38' if v>0 else '#1A7A6E' for v in top_coefs.values]
axes[0].barh(top_coefs.index[::-1], top_coefs.values[::-1],
             color=colors_c[::-1], edgecolor='white')
axes[0].axvline(0, color='black', lw=1)
axes[0].set_title('Top 12 Coefficients (log-odds)', fontweight='bold')

top_or = odds_ratios.head(top_n)
colors_or = ['#9C3D38' if v>1 else '#1A7A6E' for v in top_or.values]
```



```
axes[1].barh(top_or.index[::-1], top_or.values[::-1],
             color=colors_or[::-1], edgecolor='white')
axes[1].axvline(1, color='black', lw=1, ls='--')
axes[1].set_title('Top 12 Odds Ratios (exp(coef))', fontweight='bold')
axes[1].set_xlabel('Odds Ratio (OR > 1 → ↑ risque churn)')
plt.tight_layout() ; plt.show()
```

Interprétation de l'Odds Ratio (OR)

$OR = \exp(\theta_j)$ Si $OR=2.5 \rightarrow$ la feature x_j multiplie le risque de churn par 2.5 pour +1 unité (standardisée)

Questions d'analyse

20. Quelle feature a le plus grand Odds Ratio > 1 ? Interpréter en français : '+1 unité de [feature] multiplie le risque de churn par [OR]'.
21. Quelle feature a le plus petit Odds Ratio < 1 (effet protecteur) ? Quelle est son interprétation métier ?
22. Est-ce que **Total_Trans_Ct** (nombre de transactions) a un effet positif ou négatif sur le risque de churn ? Est-ce cohérent avec votre analyse EDA (TD3) ?
23. Pourquoi interprète-t-on l'Odds Ratio avec **des coefficients standardisés** plutôt que des coefficients bruts ?



Partie 4 — Frontières de Décision et Régularisation

Durée estimée : 25 minutes | Visualiser la géométrie de la classification et l'impact de la régularisation.

Q 4.1

Visualisation de la frontière de décision — 2D

Consigne

Réduisez à 2 features pour visualiser la frontière de décision de la Régression Logistique dans un plan 2D. Comparez visuellement avec les données réelles.

```
# Sélectionner les 2 features les plus discriminantes
feat1 = 'Total_Trans_Ct'
feat2 = 'Total_Revolving_Bal'

X_2d = df_enc[[feat1, feat2]].values
y_2d = df_enc['churn'].values

X_2d_tr, X_2d_te, y_2d_tr, y_2d_te = train_test_split(
    X_2d, y_2d, test_size=0.2, random_state=42, stratify=y_2d)

pipe_2d = Pipeline([
    ('sc', StandardScaler()),
    ('lr', LogisticRegression(max_iter=1000, C=1.0))
])
pipe_2d.fit(X_2d_tr, y_2d_tr)

# Grille pour la frontière de décision
scaler_2d = pipe_2d.named_steps['sc']
x1_min, x1_max = X_2d[:,0].min()-1, X_2d[:,0].max()+1
x2_min, x2_max = X_2d[:,1].min()-100, X_2d[:,1].max()+100
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max, 300),
                        np.linspace(x2_min, x2_max, 300))
Z = pipe_2d.predict_proba(np.c_[xx1.ravel(), xx2.ravel()])[:,1]
Z = Z.reshape(xx1.shape)

fig, ax = plt.subplots(figsize=(12, 7))
contour = ax.contourf(xx1, xx2, Z, levels=20, cmap='RdYlGn_r', alpha=.6)
plt.colorbar(contour, ax=ax, label='P(churn=1)')

# Points de données
for cls, color, label in [(0, '#1A7A6E', 'Existing'), (1, '#9C3D38', 'Attrited')]:
    mask = y_2d_te == cls
    ax.scatter(X_2d_te[mask,0], X_2d_te[mask,1],
               c=color, alpha=.6, s=20, label=label)
```



```
# Frontière de décision (P=0.5)
ax.contour(xx1, xx2, Z, levels=[0.5], colors='navy', linewidths=2.5)
ax.set_xlabel(featl1) ; ax.set_ylabel(featl2)
ax.set_title(f'Frontière de Décision – Régression Logistique\n{featl1} vs {featl2}',
            fontweight='bold')
ax.legend() ; plt.tight_layout() ; plt.show()
```

Questions d'analyse

24. La frontière de décision est-elle linéaire ou courbe ? Était-ce attendu pour la Régression Logistique ?
25. Observez la zone de transition (couleur jaune). Que représente-t-elle en termes de certitude du modèle ? Comment interprétez-vous un client qui tombe dans cette zone ?
26. Changez le couple de features (essayez **Avg_Utilization_Ratio** + **Contacts_Count_12_mon**). La frontière est-elle mieux placée ?

Q 4.2

Régularisation L1 (Lasso) et L2 (Ridge) — impact sur les coefficients

Consigne

Comparez l'effet de la régularisation L1 et L2 sur les coefficients de la Régression Logistique. Tracez le chemin de régularisation pour différentes valeurs de C.

```
# C = 1/λ : grand C = peu de régularisation, petit C = forte régularisation
C_values = np.logspace(-3, 2, 30) # De 0.001 à 100

coefs_l1 = {feat: [] for feat in X_tr.columns[:8]} # Top 8 features
coefs_l2 = {feat: [] for feat in X_tr.columns[:8]}

scaler_reg = StandardScaler()
X_tr_sc = scaler_reg.fit_transform(X_tr)
X_te_sc = scaler_reg.transform(X_te)

for C in C_values:
    # L1 (Lasso)
    lr_l1 = LogisticRegression(C=C, penalty='l1', solver='liblinear',
                              max_iter=1000, random_state=42)
    lr_l1.fit(X_tr_sc, y_tr)
    for feat in coefs_l1:
        idx = list(X_tr.columns).index(feat)
        coefs_l1[feat].append(lr_l1.coef_[0][idx])

    # L2 (Ridge)
```



```
lr_l2 = LogisticRegression(C=C, penalty='l2', solver='lbfgs',
                           max_iter=1000, random_state=42)
lr_l2.fit(X_tr_sc, y_tr)
for feat in coefs_l2:
    idx = list(X_tr.columns).index(feat)
    coefs_l2[feat].append(lr_l2.coef_[0][idx])

# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
for feat in coefs_l1:
    axes[0].plot(C_values, coefs_l1[feat], label=feat)
    axes[1].plot(C_values, coefs_l2[feat], label=feat)

for ax, title in zip(axes, ['L1 (Lasso) - Sélection de features', 'L2 (Ridge) - Réduction progressive']):
    ax.set_xscale('log') ; ax.axhline(0, color='black', lw=0.8, ls='--')
    ax.set_xlabel('C (1/λ) - plus grand = moins de régularisation')
    ax.set_ylabel('Coefficient') ; ax.set_title(title, fontweight='bold')
    ax.legend(fontsize=7, loc='best')
plt.tight_layout() ; plt.show()
```

📌 Rappel de cours — Rappel — L1 vs L2 dans la Régression Logistique

- **L1 (Lasso, penalty='l1')** : certains coefficients deviennent exactement 0 → sélection automatique de features
- **L2 (Ridge, penalty='l2')** : tous les coefficients diminuent mais aucun n'atteint exactement 0 → compression
- **C = 1/λ** : grand C → peu de régularisation (risque d'overfitting) / petit C → forte régularisation (risque d'underfitting)
- **Solver** : L1 nécessite 'liblinear' ou 'saga', L2 accepte tous les solvers

Questions d'analyse

27. Sur le graphique L1 : à partir de quelle valeur de C certains coefficients commencent-ils à être exactement 0 ? Quelles features sont éliminées en premier ?
28. Sur le graphique L2 : les coefficients atteignent-ils jamais exactement 0 ? Quelle est la différence fondamentale de comportement entre L1 et L2 ?
29. **Bonus réflexion** : pour une banque qui veut un modèle de churn simple à expliquer au régulateur (moins de features, plus lisible), quelle pénalité recommandez-vous ?

Q 4.3

Sélection du paramètre C optimal par cross-validation



```
from sklearn.model_selection import GridSearchCV

param_grid = {'model__C': [0.001, 0.01, 0.1, 0.5, 1, 5, 10, 50]}

pipe_cv = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000, random_state=42))
])

grid = GridSearchCV(
    pipe_cv, param_grid,
    cv=5, scoring='f1',      # F1 car données déséquilibrées !
    n_jobs=-1, verbose=0
)
grid.fit(X_tr, y_tr)

print(f'Meilleur C : {grid.best_params_["model__C"]}')
print(f'Meilleur F1 CV : {grid.best_score_:.4f}')

# Courbe de validation
cv_results = pd.DataFrame(grid.cv_results_)
plt.figure(figsize=(10, 5))
plt.semilogx(param_grid['model__C'], cv_results['mean_test_score'],
             'o-', color='#1A7A6E', lw=2, label='F1 moyen CV')
plt.fill_between(param_grid['model__C'],
                 cv_results['mean_test_score'] - cv_results['std_test_score'],
                 cv_results['mean_test_score'] + cv_results['std_test_score'],
                 alpha=.2, color='#1A7A6E')
plt.axvline(grid.best_params_['model__C'], color='#9C3D38',
            ls='--', label=f'C optimal = {grid.best_params_["model__C"]}')
plt.xlabel('C (1/λ)') ; plt.ylabel('F1 Score (CV)')
plt.title('Courbe de Validation – Régression Logistique', fontweight='bold')
plt.legend() ; plt.show()

# Évaluation finale du meilleur modèle
y_best = grid.best_estimator_.predict(X_te)
print(classification_report(y_te, y_best,
                           target_names=['Existing (0)', 'Attrited (1)']))
```

Questions d'analyse

30. Quelle valeur de C est sélectionnée par la cross-validation ? Correspond-elle à une forte ou faible régularisation ?
31. Pourquoi utilise-t-on `scoring='f1'` dans GridSearchCV plutôt que `scoring='accuracy'` sur ce dataset ?
32. Le F1-score sur le test set avec C optimal est-il meilleur qu'avec C=1 (défaut sklearn) ? De combien ?



Section Bonus — Pour aller plus loin

Ces exercices approfondissent des aspects avancés de la régression, utiles pour les projets réels.

Bonus B1 — Régression Logistique Polynomiale : briser la linéarité

La frontière de décision de la Régression Logistique est linéaire dans l'espace des features. En ajoutant des features polynomiales, on peut créer des frontières non-linéaires.

```
from sklearn.preprocessing import PolynomialFeatures

# Pipeline avec features polynomiales de degré 2
pipe_poly = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('model', LogisticRegression(C=1.0, max_iter=1000, random_state=42))
])

# Entraîner sur les 2 features du Q4.1
pipe_poly.fit(X_2d_tr, y_2d_tr)

print(f'Score linéaire : {pipe_2d.score(X_2d_te, y_2d_te):.4f}')
print(f'Score poly deg2 : {pipe_poly.score(X_2d_te, y_2d_te):.4f}')
print(f'Nb de features après poly :
{pipe_poly["poly"].get_feature_names_out().shape[0]}')
```

33. La frontière polynomiale est-elle plus courbe ? Comment visualiseriez-vous cela ?

34. Avec degree=3 et peu de données, quel risque courez-vous ? Comment le détecter ?

Bonus B2 — Matrice de Confusion Normalisée et Analyse d'Erreurs

```
# Matrice de confusion normalisée
from sklearn.metrics import ConfusionMatrixDisplay

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, normalize, title in zip(
    axes,
    [None, 'true'],
    ['Matrice de Confusion (comptes)', 'Matrice de Confusion (normalisée)']
):
    cm = confusion_matrix(y_te, y_pred_log, normalize=normalize)
    disp = ConfusionMatrixDisplay(cm,
                                  display_labels=['Existing', 'Attrited'])
    disp.plot(ax=ax, colorbar=False,
```



```
        cmap='Blues' if normalize else 'YlOrRd')
    ax.set_title(title, fontweight='bold')
plt.tight_layout() ; plt.show()

# Analyse des faux négatifs (churns non détectés)
false_negatives = X_te[(y_te==1) & (y_pred_log==0)]
false_negatives_df = pd.DataFrame(false_negatives, columns=X_tr.columns)
print('\nProfil moyen des churns non détectés (Faux Négatifs) :')
print(false_negatives_df[['Total_Trans_Ct', 'Total_Revolving_Bal',
                          'Avg_Utilization_Ratio']].mean().round(3))
```

35. Que représente chaque case de la matrice normalisée (valeurs entre 0 et 1) ?

36. Le profil moyen des Faux Négatifs est-il différent du profil moyen des vrais Attrited ? Qu'est-ce que cela indique sur les limites du modèle ?



Livrable attendu

✓ Ce que vous devez remettre

- Notebook **TD4_NomPrenom.ipynb** — toutes les cellules exécutées avec outputs visibles
- **Implémentation NumPy from scratch** du Gradient Descent avec test de plusieurs learning rates
- **Régression Linéaire Multiple** : tableau des coefficients standardisés interprétés
- **Régression Logistique** avec courbe ROC, seuil optimal justifié, Odds Ratios commentés
- **Visualisation de la frontière de décision** sur au moins 2 couples de features
- **Chemin de régularisation L1 et L2** — analyse des features sélectionnées ou compressées
- Une cellule Markdown de **conclusion comparative** (150 mots) : Régression Linéaire vs Logistique, leçons tirées
- Délai : **avant le TD 5**, dépôt sur la plateforme pédagogique

Critères d'évaluation

Critère	Coefficient	Description
Implémentation Gradient Descent (exactitude + commentaires)	20%	Code numpy correct, tests de convergence
Interprétation des coefficients (linéaire + logistique)	25%	Lien coefficients $\leftrightarrow$ contexte métier
Choix et justification du seuil de décision	20%	Argumentaire précision/rappel/métier
Analyse de la régularisation L1/L2	25%	Chemin, sélection de C, impact observé
Section Bonus (optionnel)	+ 10%	Poly features ou analyse Faux Négatifs

Ressources et Documentation

- **sklearn — LinearRegression** : https://scikit-learn.org/stable/modules/linear_model.html
- **sklearn — LogisticRegression** : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- **sklearn — Métriques de classification** : https://scikit-learn.org/stable/modules/model_evaluation.html
- **Cours Stanford — Régression Logistique** : https://cs229.stanford.edu/notes2023fall/main_notes.pdf



- **Gradient Descent visualisé** : <https://ml-concepts.netlify.app/>

Paramètres clés de LogisticRegression à retenir

Paramètre	Valeur défaut	Rôle
C	1.0	Inverse de la régularisation. Petit C = fort régularisation
penalty	'l2'	Type de régularisation : 'l1', 'l2', 'elasticnet', 'none'
solver	'lbfgs'	Algorithme d'optimisation. 'liblinear' requis pour L1 petit dataset
max_iter	100	Itérations max GD. Augmenter si ConvergenceWarning
class_weight	None	'balanced' pour données déséquilibrées (alternative à SMOTE)
multi_class	'auto'	'ovr' ou 'multinomial' pour classification multi-classes